

PGC308A – Estimativa de Conformidade a SLAs

Relatório Final – 2026/1

Victor Guimarães Silva

Raquel de Fátima Alves

PPGCO/FACOM – Universidade Federal de Uberlândia

Junho de 2026

Resumo

Este relatório descreve a implementação e análise de modelos de aprendizado de máquina supervisionado para estimar a conformidade a SLAs em um serviço de vídeo sob demanda (VoD). O problema consiste em prever, a partir de métricas do servidor Linux, se o serviço atende ao SLA definido como taxa mínima de frames no cliente (≥ 18 fps). Foram implementadas três tasks: exploração dos dados (Task I), regressão linear para estimativa contínua dos fps (Task II) e regressão logística para classificação binária de conformidade (Task III). Como atividade bônus, aplicou-se regressão linear à predição de qualidade de sinal 5G em traces coletados em campo em São Paulo. O código-fonte completo está disponível em: <https://github.com/victorguimaraessilva/Trabalho-IA-MSc>.

Sumário

1	Introdução	3
2	Dados e Metodologia	3
2.1	Descrição dos Dados	3
2.2	Protocolo Experimental	4
3	Task I – Exploração dos Dados	4
3.1	Estatísticas Descritivas	4
3.2	Consultas Específicas	5
3.3	Visualizações	5

4	Task II – Regressão Linear	6
4.1	Objetivo e Métrica	6
4.2	Implementação	6
4.3	Coeficientes do Modelo M (treino completo)	6
4.4	Resultados por Tamanho de Treino	7
4.5	Análise no Conjunto de Teste	8
5	Task III – Regressão Logística	8
5.1	Objetivo e Métrica	8
5.2	Implementação	8
5.3	Balanceamento das Classes	9
5.4	Resultados	9
5.5	Visualizações	10
6	Atividade Bônus – 5G Datasets Challenge (Atividade A)	10
6.1	Descrição	10
6.2	Resultados	11
7	Conclusão	11

1 Introdução

Provedores de serviços de vídeo sob demanda (VoD) precisam garantir que os usuários recebam qualidade satisfatória de reprodução. O *Service Level Agreement* (SLA) formaliza esse compromisso: neste trabalho, o SLA é definido como a taxa de frames percebida pelo cliente (**DispFrames**) igual ou superior a 18 fps. Medir diretamente a qualidade no terminal do usuário em tempo real é, em geral, inviável em escala. A alternativa estudada neste trabalho é usar **aprendizado de máquina supervisionado** para inferir a qualidade do cliente a partir de métricas coletadas no próprio servidor – uma prática conhecida como *service assurance*.

O conjunto de dados contém 3.600 observações, uma por segundo, durante uma hora de operação. As variáveis de entrada (X) descrevem o estado do servidor Linux (CPU, memória, processos, rede, etc.) e a variável alvo (Y) é a taxa de frames observada no cliente (**DispFrames**).

Três tasks foram propostas: (I) exploração e análise dos dados; (II) regressão linear para prever os fps com métrica NMAE; e (III) regressão logística para classificar o serviço como *conforme* ou *em violação* do SLA com métrica ERR. Adicionalmente, foi implementada a Atividade A do 5G Datasets Challenge como bônus.

O repositório com todos os notebooks e código-fonte está disponível publicamente em:

<https://github.com/victorguimaraessilva/Trabalho-IA-MSc>

2 Dados e Metodologia

2.1 Descrição dos Dados

O dataset é composto por dois arquivos:

- **X.csv**: 9 métricas do servidor Linux, amostradas a cada segundo;
- **Y.csv**: taxa de frames no cliente (**DispFrames**, em fps), no mesmo instante.

A Tabela 1 descreve cada variável de entrada.

Tabela 1: Variáveis de entrada (servidor) e variável alvo (cliente).

Variável	Significado	Unidade
<code>all_..idle</code>	CPU ociosa	%
<code>X..memused</code>	Memória RAM usada	%
<code>proc.s</code>	Taxa de criação de processos	proc/s
<code>cswch.s</code>	Trocas de contexto	trocas/s
<code>file.nr</code>	File handles abertos	contagem
<code>sum_intr.s</code>	Taxa de interrupções	int/s
<code>ldavg.1</code>	Média de carga (1 min)	adimensional
<code>tcpsck</code>	Sockets TCP em uso	contagem
<code>pgfree.s</code>	Liberação de páginas	páginas/s
<code>DispFrames</code>	Taxa de frames no cliente (alvo)	fps

2.2 Protocolo Experimental

Em todas as tasks, foi adotado o seguinte protocolo:

- **Divisão treino/teste:** 70% treino (2.520 obs.) e 30% teste (1.080 obs.), seleção uniformemente aleatória com `random_state=42`;
- **Modelos:** regressão linear e logística do `scikit-learn`;
- **SLA:** serviço *conforme* quando `DispFrames` ≥ 18 fps.

O trecho abaixo ilustra a configuração central do projeto:

Listing 1: Constantes centrais do projeto (`src/config.py`).

```

1 SLA_THRESHOLD_FPS = 18.0    # fps -- limiar de conformidade
2 TRAIN_FRACTION    = 0.70    # 70% treino
3 RANDOM_STATE      = 42      # reproducibilidade
4 TRAIN_SIZES       = [50, 500, 1000, 1500, 2520]
```

3 Task I – Exploração dos Dados

3.1 Estatísticas Descritivas

A Tabela 2 apresenta as estatísticas descritivas das variáveis mais relevantes.

Tabela 2: Estatísticas descritivas das principais variáveis (3.600 obs.).

Variável	Média	Mín.	P25	P90	Desvio Padrão
CPU ociosa (%)	9,1	0,0	4,5	19,2	8,4
Memória usada (%)	88,9	73,0	87,2	92,3	3,8
Sockets TCP	46,2	39,0	44,0	52,0	4,3
DispFrames	18,8	0,0	16,4	25,2	5,2

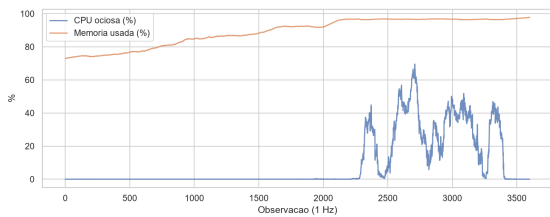
A média de **DispFrames** é 18,8 fps, muito próxima do limiar de SLA (18 fps), indicando que o serviço opera frequentemente na fronteira da conformidade. A memória usada permanece acima de 73% em todos os instantes, evidenciando um servidor consistentemente carregado.

3.2 Consultas Específicas

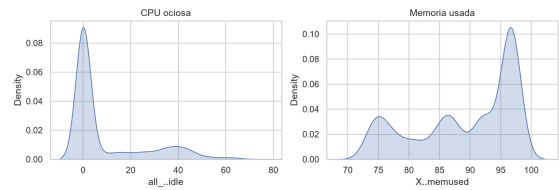
- (a) **Observações com memória > 80%:** 2.875 (79,9% do trace).
- (b) **Média de sockets TCP quando interrupções > 18.000/s:** 46,35.
- (c) **Mínimo de memória quando CPU ociosa < 20%:** 73,03%.

O resultado (a) confirma que o servidor raramente opera com memória baixa. O resultado (c) mostra que, mesmo sob alta carga de CPU, a memória se mantém elevada, sugerindo que ambos os recursos são pressionados simultaneamente.

3.3 Visualizações



(a) Série temporal de CPU ociosa e memória usada.



(b) KDE de CPU ociosa e memória usada.

Figura 1: Comportamento das métricas do servidor ao longo do trace (Task I).

A Figura 1a mostra que a CPU ociosa apresenta grande variabilidade ao longo do tempo, com picos e vales frequentes, enquanto a memória permanece estável e elevada. O KDE (Figura 1b) revela que a CPU ociosa possui distribuição multimodal (com concentração entre 0–10%), ao passo que a memória tem distribuição estreita centrada em torno de 89%, confirmando o comportamento observado nas estatísticas.

4 Task II – Regressão Linear

4.1 Objetivo e Métrica

O objetivo é estimar `DispFrames` (valor contínuo) a partir das 9 variáveis do servidor. A métrica de avaliação é o **NMAE** (Normalized Mean Absolute Error):

$$\text{NMAE} = \frac{1}{\bar{y}} \cdot \frac{1}{m} \sum_{i=1}^m |y_i - \hat{y}_i| \quad (1)$$

onde $\bar{y}$ é a média de `DispFrames` no conjunto de teste e $m = 1.080$. O critério do enunciado é $\text{NMAE} < 15\%$.

4.2 Implementação

Listing 2: Treinamento e avaliação do modelo de regressão linear.

```
1 from sklearn.linear_model import LinearRegression
2 from src.metrics import nmae
3
4 model = LinearRegression()
5 model.fit(X_train, y_train)
6 y_pred = model.predict(X_test)
7
8 error = nmae(y_test, y_pred)    # NMAE normalizado pela media
    de y_test
```

4.3 Coeficientes do Modelo M (treino completo)

Tabela 3: Coeficientes do modelo de regressão linear treinado com 2.520 observações.

Variável	Coefficiente (Θ)
CPU ociosa (%)	−0,0921
Memória usada (%)	−0,0868
Média de carga — 1 min	−0,0606
Sockets TCP em uso	−0,0653
Taxa de criação de processos	−0,0120
File handles em uso	−0,0031
Taxa de trocas de contexto	≈ 0
Taxa de interrupções	≈ 0
Taxa de liberação de páginas	≈ 0
Intercepto (Θ_0)	49,69

Todos os coeficientes significativos são negativos, indicando que maiores valores de pressão no servidor (memória ocupada, carga de CPU, sockets ativos) estão associados a estimativas menores de taxa de frames. O intercepto de 49,69 representa o valor esperado de `DispFrames` quando todas as variáveis estão zeradas – um valor de referência sem interpretação física direta, dado que as variáveis não assumem valor nulo na prática.

4.4 Resultados por Tamanho de Treino

Tabela 4: NMAE e tempo de treino em função do tamanho do conjunto de treino.

Obs. de treino	Tempo (ms)	NMAE (%)	Acurado?
50	1,1	11,52	Sim
500	1,9	10,58	Sim
1.000	1,2	10,49	Sim
1.500	1,2	10,38	Sim
2.520	2,6	10,39	Sim

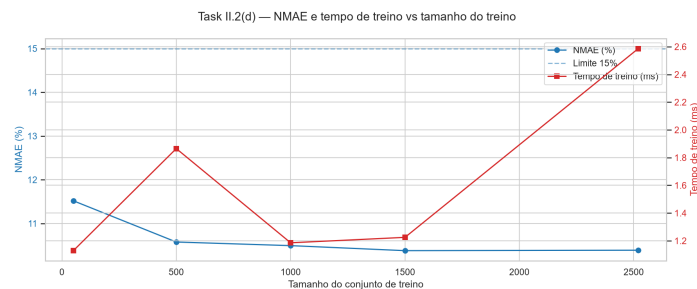
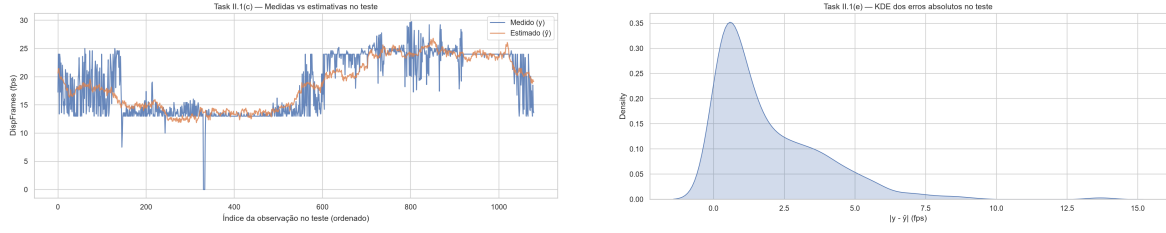


Figura 2: NMAE e tempo de treino em função do número de observações de treino (Task II).

A Figura 2 mostra que, mesmo com apenas 50 amostras, o modelo já atinge $\text{NMAE} = 11,52\%$, abaixo do limiar de 15%. O ganho de acurácia ao aumentar o treino de 500 para 2.520 observações é inferior a 0,2 pp, caracterizando **retornos decrescentes**. O tempo de treino permanece sub-milissegundo a poucos ms, irrelevante para a decisão de tamanho.

4.5 Análise no Conjunto de Teste



(a) Série temporal: medido vs. estimado.

(b) KDE dos erros absolutos $|y - \hat{y}|$.

Figura 3: Avaliação do modelo de regressão linear no conjunto de teste (Task II).

A série temporal (Figura 3a) mostra que o modelo acompanha a tendência geral dos fps, errando mais nos picos e vales – onde a taxa de frames muda abruptamente. O KDE dos erros absolutos (Figura 3b) confirma que a maioria dos erros se concentra abaixo de 5 fps, com cauda longa em eventos extremos. O NMAE do modelo completo é **10,39%**, abaixo do limite de 15%.

5 Task III – Regressão Logística

5.1 Objetivo e Métrica

Em vez de prever os fps exatos, o objetivo aqui é classificar cada instante como:

- **Classe 1 (conforme):** $\text{DispFrames} \geq 18$ fps;
- **Classe 0 (violação):** $\text{DispFrames} < 18$ fps.

A métrica é o **ERR** (taxa de classificação errada):

$$\text{ERR} = 1 - \frac{VP + VN}{m} \quad (2)$$

onde VP = verdadeiros positivos, VN = verdadeiros negativos e $m = 1.080$. O critério é $\text{ERR} < 15\%$.

5.2 Implementação

Listing 3: Criação dos rótulos e treinamento do classificador.

```
1 from sklearn.linear_model import LogisticRegression
2 from src.metrics import classification_error, sla_labels
3
4 # binariza Y: 1 se conforme, 0 se violacao
```



```

5 y_bin = sla_labels(y, SLA_THRESHOLD_FPS)
6
7 clf = LogisticRegression(max_iter=10000, random_state=
    RANDOM_STATE)
8 clf.fit(X_train, y_bin_train)
9 y_pred = clf.predict(X_test)
10
11 err = classification_error(y_bin_test, y_pred)

```

5.3 Balanceamento das Classes

No split de treino, aproximadamente 52% das observações são conformes e 48% em violação. No conjunto de teste, a proporção é 50/50. O balanceamento razoável entre as classes valida o uso de ERR como métrica primária.

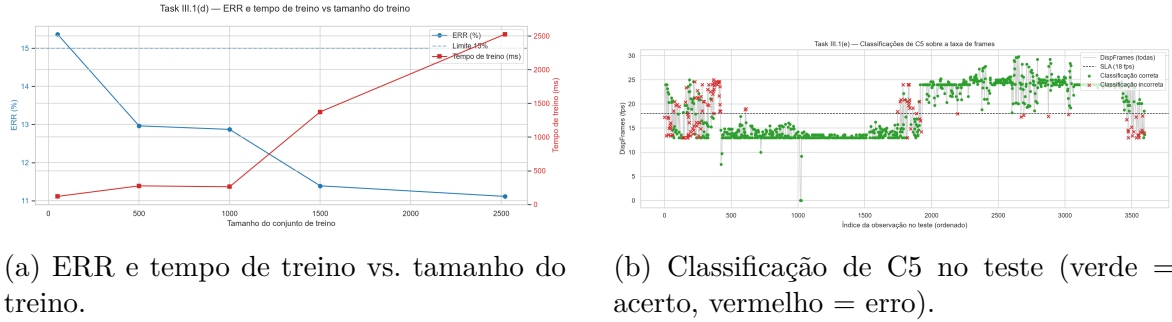
5.4 Resultados

Tabela 5: Resultados dos classificadores C1–C5 no conjunto de teste.

Classif.	Treino	Tempo (ms)	ERR (%)	Acurado?	VP	VN	FP	FN
C1	50	119	15,37	Não	438	476	59	107
C2	500	275	12,96	Sim	465	475	60	80
C3	1.000	262	12,87	Sim	464	477	58	81
C4	1.500	1373	11,39	Sim	479	478	57	66
C5	2.520	2527	11,11	Sim	486	474	61	59

C1 (50 amostras) não atinge o critério de 15%, ficando no limite. A partir de C2 (500 amostras), todos os classificadores são acurados. C5 obtém o melhor ERR: **11,11%**. O FN (serviço classif. como conforme quando viola) cai de 107 (C1) para 59 (C5), o que é importante do ponto de vista operacional – falsos negativos levam o provedor a não agir em situações de violação real.

5.5 Visualizações



(a) ERR e tempo de treino vs. tamanho do treino.

(b) Classificação de C5 no teste (verde = acerto, vermelho = erro).

Figura 4: Resultados da regressão logística (Task III).

A Figura 4a confirma a tendência decrescente do ERR com mais dados, com ganho expressivo entre 50 e 500 amostras e retornos decrescentes a partir daí. O tempo de treino cresce mais acentuadamente de 1.500 para 2.520 amostras, refletindo a natureza iterativa da regressão logística.

A Figura 4b mostra que os erros do melhor classificador (C5) se concentram próximos ao limiar de 18 fps – região onde a separação entre as classes é intrinsecamente difícil. Longe do limiar, o modelo classifica corretamente com alta consistência.

6 Atividade Bônus – 5G Datasets Challenge (Atividade A)

6.1 Descrição

Como atividade opcional, foi implementada a **Atividade A** do 5G Datasets Challenge (SMARTNESS/UNICAMP): predição de qualidade de sinal (Qual/RSRQ, em dB) a partir de métricas de rede coletadas com Samsung S21 5G na rede Claro em São Paulo.

Foram utilizados três traces do dataset **g-nettrack-pro**:

Tabela 6: Traces utilizados na Atividade Bônus.

Arquivo	Cenário
2023-01-21_308-walking-paulista-1.txt	Pedestre – Av. Paulista
2023-01-22_933-driving-sp-1.txt	Veículo em deslocamento
2023-01-21_244-subway-1.txt	Metrô

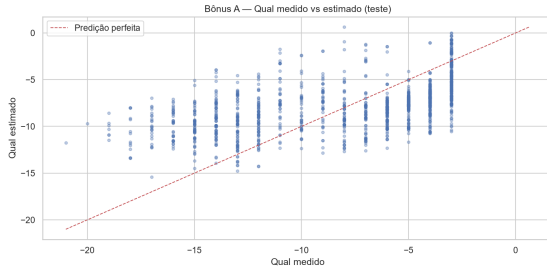
As features utilizadas foram: **Level** (RSRP, dBm), **SNR**, **DL_bitrate** (kbps), **UL_bitrate** (kbps) e **Speed** (km/h). Após remoção de valores inválidos (-, -99), restaram ≈ 4.500 registros. O mesmo protocolo de split 70/30 com **random_state=42** foi aplicado.

6.2 Resultados

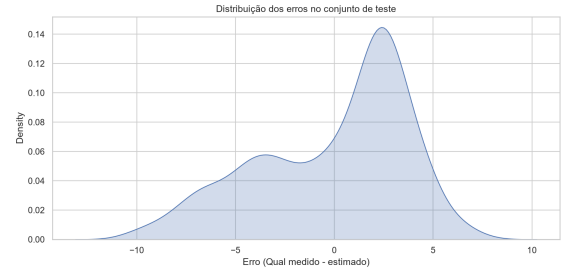
Tabela 7: Métricas do modelo de regressão linear para predição de Qual (5G).

Métrica	Valor	Interpretação
MAE	3,20 dB	Erro médio absoluto entre Qual medido e estimado
RMSE	3,80 dB	Raiz do EQM – penaliza erros grandes
R^2	0,29	Fração da variância de Qual explicada pelo modelo

O coeficiente de determinação $R^2 = 0,29$ indica relação parcial – esperado para um modelo linear simples aplicado a dados heterogêneos que combinam cenários de pedestre, veículo e metrô (indoor/outdoor), com eventos de handover não modelados. A variável SNR é a de maior influência ($\Theta = +0,346$): maior relação sinal-ruído está associada a melhor qualidade de sinal estimada.



(a) Qual medido vs. estimado.



(b) KDE dos erros de estimativa.

Figura 5: Avaliação do modelo de predição de qualidade de sinal 5G (Atividade Bônus).

O scatter plot (Figura 5a) mostra dispersão considerável em torno da diagonal, confirmando R^2 moderado. O KDE dos erros (Figura 5b) apresenta distribuição aproximadamente simétrica centrada em zero, sem viés sistemático de superestimação ou subestimação – o modelo erra de forma equilibrada, o que é desejável.

7 Conclusão

Este trabalho demonstrou a viabilidade de usar modelos lineares supervisionados para estimar conformidade a SLAs em serviços VoD a partir de métricas do servidor.

Na Task II, a regressão linear atingiu NMAE de **10,39%** (abaixo do limite de 15%), com retornos decrescentes a partir de 500 amostras de treino. Na Task III, o melhor classificador logístico (C5) obteve ERR de **11,11%**, com os erros concentrados na fronteira de 18 fps – região inerentemente ambígua.

As principais limitações são: (i) a linearidade dos modelos, que pode não capturar relações não lineares; (ii) o split aleatório, que pode misturar padrões temporais; e (iii) a ausência de features de rede e do terminal do cliente.

Como trabalho futuro, propõe-se o uso de modelos não lineares (Random Forest, XGBoost), validação temporal e enriquecimento das features com métricas de rede.

Na atividade bônus, a mesma metodologia aplicada a traces 5G obteve $R^2 = 0,29$, resultado modesto mas interpretável para um baseline linear com dados de campo heterogêneos.

O código-fonte completo, notebooks reproduzíveis e dados estão disponíveis em: <https://github.com/victorguimaraessilva/Trabalho-IA-MSc>